

중고거래 물건, 얼마에 팔고 배송비는 누가 낼까?

Mercari 중고거래 데이터로 살펴본 인공지능(AI) 가격·배송비 예측

분석 보고서

작성자: 정찬성

작성일: 2026년 8월 27일

대상 노트북: 11_캐글_mercari_gradientboost_rmsle_aic_bic.ipynb (전체 1,482,535건 기준)

1. 서론

1.1 연구 배경 및 필요성

중고거래 플랫폼에서는 판매자가 물건 가격을 얼마로 매겨야 할지 몰라 어려움을 겪는 경우가 많고, 배송비를 구매자와 판매자 중 누가 부담할지도 매물마다 제각각이다.

미국·일본의 대표적인 중고거래 앱인 Mercari(메루카리)에서도 판매자들이 똑같은 고민을 합니다. 그래서 이번 분석은 “상품 이름, 카테고리, 브랜드, 상태, 설명글” 같은 정보만 있으면 인공지능이 “이 정도 가격이 적당해요”, “배송비는 보통 이렇게 부담해요”라고 알려줄 수 있는지 확인해 보는 것을 목표로 했습니다.

이번 분석에는 실제 판매 기록 148만 건이 넘는 데이터 전체(mercari_train.tsv, 1,482,535건)를 사용했습니다.

또한 “얼마나 잘 맞았는가(RMSLE)”라는 점수 하나만으로는 “이 모델이 적당히 똑똑한 건지, 아니면 문제만 억지로 외운 건 아닌지”까지는 알 수 없습니다. 그래서 AIC·BIC라는 보조 점수도 함께 계산해서, 모델이 너무 복잡해지지 않는지 추가로 살펴봤습니다.

1.2 과제 목적

본 분석은 공통된 상품 정보(상품명, 카테고리, 브랜드, 상품 상태, 설명글) 위에서 두 가지 예측 모델을 만드는 것을 목적으로 합니다.

- 가격(price)을 맞추는 모델 — 물건 정보를 보고 적정 판매 가격을 예측(회귀모델)
- 배송비 부담 주체(shipping)를 맞추는 모델 — 구매자와 판매자 중 누가 배송비를 낼지 예측(분류모델)

아울러 나무(트리) 개수를 늘려가며 AIC·BIC로 “어디까지 늘리는 게 적당한가”를 확인하는 실험도 함께 진행했습니다. 다만 이 실험은 조건 하나를 확인하는 데만 4시간 가까이 걸려서, 이번 보고서 시점까지 모든 조건을 다 마치지지는 못했습니다. 미완료 상태 그대로 솔직하게 적었습니다.

2. 데이터셋 소개

2.1 데이터 출처 및 구성

데이터 분석 대회 사이트인 Kaggle에 올라와 있는 “Mercari Price Suggestion Challenge”의 판매

기록 파일(mercari_train.tsv)을 사용했습니다. 표본을 따로 뽑지 않고 전체 148만 2,535건을 그대로 분석에 사용했습니다.

항목	값
전체 데이터 건수	1,482,535건
전체 항목(컬럼) 수	8개 (식별번호 1 + 가격 1 + 배송비부담 1 + 나머지 정보 5)
맞혀야 할 것 ①	price — 실제 판매 가격 (숫자로 예측)
맞혀야 할 것 ②	shipping — 배송비를 누가 내는지 (구매자/판매자, 둘 중 하나로 예측)
결과 확인용으로 따로 댄 건수	296,507건 (전체의 20%)

2.2 데이터 특성 및 구조 분석

이번 데이터는 숫자만 있는 게 아니라 글(상품 이름, 상품 설명), 브랜드 이름, 여러 단계로 나뉜 카테고리, 순서가 있는 상품 상태 등급처럼 성격이 다른 정보들이 뒤섞여 있습니다. 상품 이름과 브랜드에는 매우 다양한 단어가 등장해, 텍스트를 숫자로 바꾸는 과정에서 항목 수가 크게 늘어납니다(자세한 내용은 2.7절에 추가 기재).

2.3 Feature 이름 특징

데이터 안의 8개 항목은 모두 이름만 봐도 무슨 뜻인지 알 수 있는 것들입니다.

항목 이름	설 명
train_id	물건 하나하나에 매겨진 고유 번호
name	상품 이름
item_condition_id	상품 상태 등급 (1~5, 숫자가 클수록 상태가 나쁨)
category_name	카테고리 (“대분류/중분류/소분류” 형식)
brand_name	브랜드 이름 (null 값 엄청 43%)

항목 이름	설 명
price	실제 판매 가격 ->이번에 맞히고 싶은 값 ①
shipping	배송비 부담 주체(0/1) ->이번에 맞히고 싶은 값 ②
item_description	상품 상세 설명

2.4 브랜드 결측 및 카테고리 계층 처리

brand_name(브랜드) 항목은 개인 판매자가 브랜드를 적지 않은 경우가 전체의 약 43%나 될 만큼 비어 있는 경우가 많습니다. 그런데 “브랜드를 적었는가, 안 적었는가” 그 자체가 가격을 가늠하는 중요한 단서였습니다(3.2절 피쳐 중요도 1위). 그래서 이 항목을 버리지 않고, 비어 있는 칸을 ‘브랜드 없음’이라는 값으로 채워서 그대로 모델에 넣었습니다. 카테고리도 마찬가지로 빈 칸을 ‘브랜드 없음’과 같은 방식으로 채운 뒤, “대분류·중분류·소분류” 3개 항목으로 나눠서 사용했습니다. 확인해 보니 이 6개 항목 모두 빈 칸이 남김없이 채워졌습니다.

2.7 벡터화로 인한 고차원·희소 피쳐 구성

컴퓨터는 “나이키”, “귀엽다” 같은 글자를 그대로 이해하지 못합니다. 그래서 상품 이름과 설명에 등장하는 단어 하나하나를 “이 단어가 등장했는지, 몇 번 등장했는지”를 나타내는 숫자로 바꿔주는 과정을 거칩니다. 이 과정을 거치면 항목(피쳐) 개수가 크게 늘어나는데, 이번 데이터에서는 그 개수가 약 16만 2천 개에 이르렀습니다. 그중에서도 상품 이름에서 뽑아낸 단어 종류가 가장 큰 비중을 차지했습니다.

정보 조각	숫자로 바꾼 방식	항목 수
상품 설명	단어 중요도 점수화 (1~3개 단어 묶음, 최대 5만 개)	50,000개
상품 이름	단어 등장 횟수 세기	105,757개
브랜드·상태·카테고리 5종	있음/없음으로 펼치기	5,811개
전체 합계 (shipping 제외)	위 조각들을 옆으로 이어붙임	161,568개

2.8 ID 변수

train_id는 물건마다 붙는 단순한 순번이라 가격이나 배송비를 예측하는 데는 아무 도움이 되지 않습니다. 그래서 처음부터 모델에 넣는 항목 목록에서 제외했습니다.

2.9 타겟 변수 분포 분석

가격(price)은 저렴한 물건이 대부분이고 아주 비싼 물건은 소수인, 한쪽으로 쏠린 분포를 보입니다. 배송비(shipping)는 검증용으로 떼어 둔 296,507건 기준으로 구매자가 부담하는 경우가 55.3%(163,887건), 판매자가 부담하는 경우가 44.7%(132,620건)로 나타났습니다.

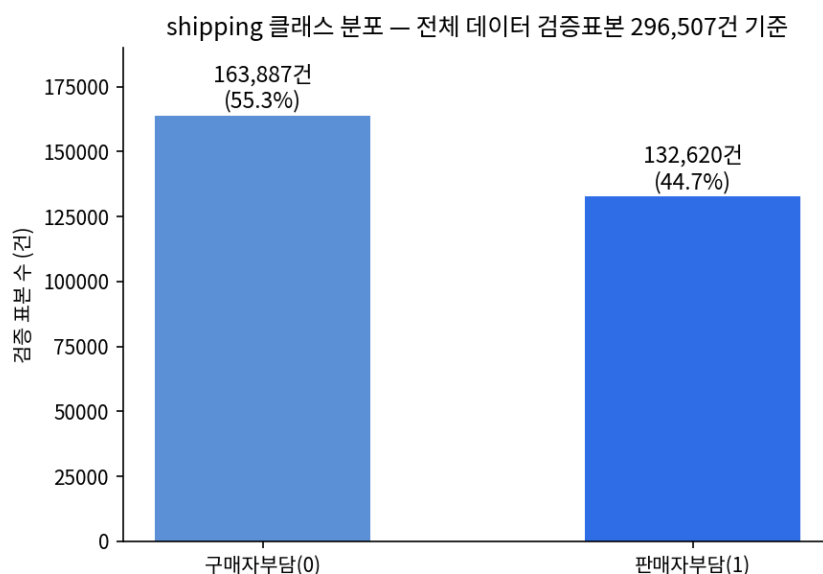


그림 2-1. 배송비 부담 주체 분포 (검증용으로 떼어 둔 296,507건 기준)

3. 탐색적 데이터 분석(EDA)

3.1 기술통계 분석

이번 데이터는 글과 범주가 섞인 데이터라서, 흔히 말하는 평균·표준편차보다는 “각 정보 조각이 숫자로 바뀐 뒤 크기가 얼마나 되는가”를 확인하는 방식으로 점검했습니다.

정보 조각	숫자로 바꾼 방식	결과 크기 (건수 × 항목 수)
상품 이름	단어 등장 횟수 세기	1,482,535 × 105,757
상품 설명	단어 중요도 점수화(1~3개 단어 묶음)	1,482,535 × 50,000
브랜드·상태·카테고리 5종	있음/없음으로 펼치기	1,482,535 × 5,811

정보 조각	숫자로 바꾼 방식	결과 크기 (건수 × 항목 수)
최종 합친 표(shipping 제외)	위 조각들을 옆으로 이어붙임	1,482,535 × 161,568

3.2 가격 예측 피쳐 중요도 분석

모델이 실제로 어떤 정보를 가장 많이 참고해서 가격을 맞췄는지 확인해 봤습니다. 브랜드를 적었는지 여부와 상품 이름 속 특정 브랜드 단어가 가장 큰 영향을 줬고, 신발·여성가방 같은 카테고리도 가격을 크게 나누었습니다.

순위	어떤 정보인가	구체적인 값
1	브랜드를 적었는지 여부	브랜드없음
2	상품 이름 속 단어	"lularoe" (인기 의류 브랜드명)
3	중분류 카테고리	Shoes(신발)
4	중분류 카테고리	Women's Handbags(여성 가방)
5	상품 설명 속 단어	"box"(박스 포함)
6	소분류 카테고리	Cell Phones & Smartphones(휴대폰)
7	상품 설명 속 단어	"authentic"(정품)
8	상품 이름 속 단어	"bundle"(묶음판매)
9	대분류 카테고리	Beauty(미용)
10	브랜드	Lululemon

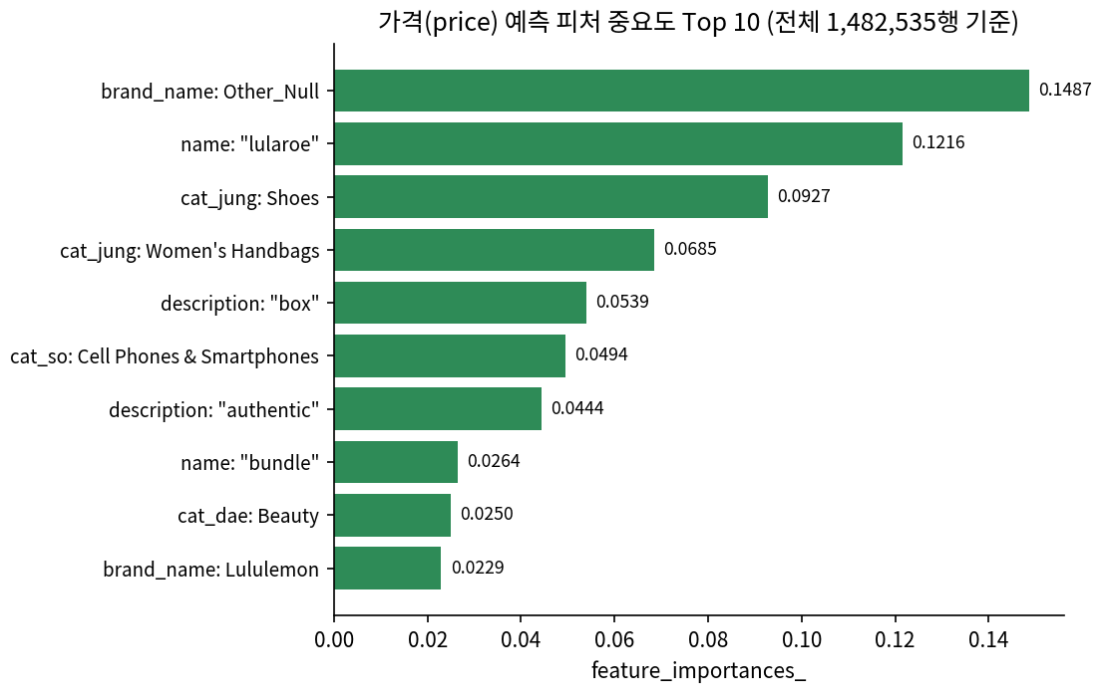


그림 3-1. 가격 예측에 가장 큰 영향을 준 정보 Top 10 (전체 1,482,535건 기준)

브랜드를 적지 않은 경우와 상품 이름에 "lularoe"가 들어간 경우가 1·2위를 차지했고, 신발·여성가방 같은 카테고리가 가격을 크게 가른다는 점도 확인됐습니다. 미용 카테고리나 룰루레몬 브랜드처럼 자주 등장하지 않는 세부 패턴까지 안정적으로 잡아낸 점도 눈에 띕니다.

3.3 이상치 및 특이값 분석

브랜드·카테고리·상품설명이 비어 있는 경우는 모두 동일하게 '브랜드없음' 같은 값으로 채워 넣었습니다. 가격이 한쪽으로 쏠려 있는 것도 넓게 보면 특이값 문제로 볼 수 있는데, 이는 3.6절에서 설명하는 로그 변환으로 완화했습니다.

3.4 분류 성능의 클래스별 분해 시각화

앞서 본 평균 점수만으로는 구매자부담·판매자부담 중 어느 쪽을 더 잘 맞히는지 보이지 않습니다. 그래서 두 그룹을 따로 떼어서 다시 확인했습니다.

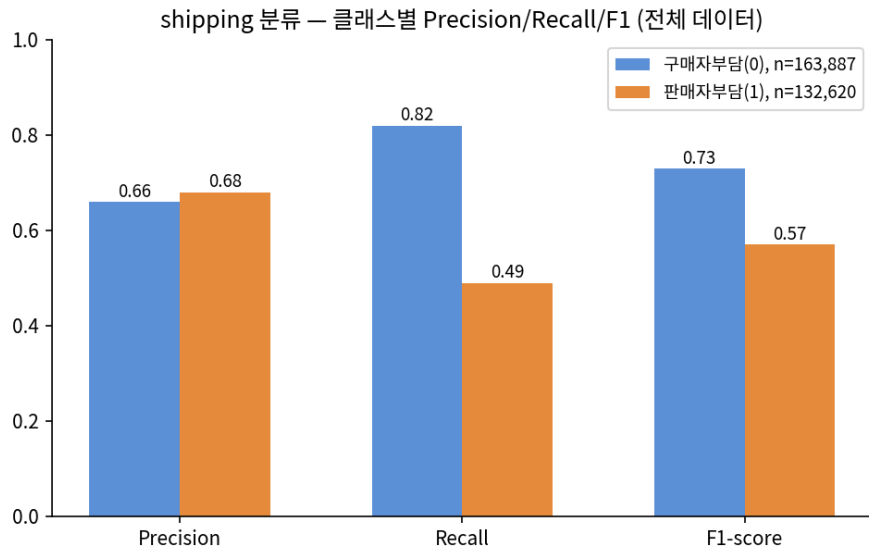


그림 3-2. 배송비 부담 주체 예측 — 구매자부담(0) vs 판매자부담(1) 세부 점수 비교

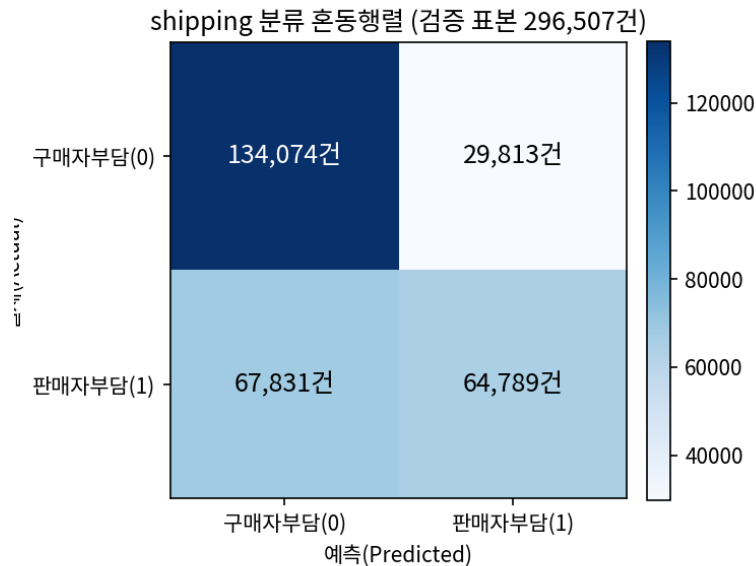


그림 3-3. 배송비 부담 주체 예측 결과표 (검증용 296,507건 기준)

구매자가 부담하는 경우는 82%를 제대로 맞췄지만, 판매자가 부담하는 경우는 49%밖에 맞히지 못했습니다. 결과표에서도 “실제로는 판매자가 냈는데 구매자가 낸 것으로 잘못 예측”한 경우가 67,831건으로, 그 반대(29,813건)보다 두 배 넘게 많습니다. 판매자부담 케이스를 더 잘 잡아내려면 추가적인 보완이 필요해 보입니다.

3.5 벡터화 피쳐 구성 확정

2.7절에서 만든 7개 조각(브랜드, 카테고리 3종, 상품 이름, 상품 설명, 상태등급)을 옆으로 이어붙여 최종 표(1,482,535행 × 161,568열)를 완성했습니다. 배송비(shipping)는 배송비 예측 모델에서

맞혀야 할 정답이므로, 정답을 미리 알려주는 셈이 되지 않도록 가격을 맞추는 모델에서도 일부러 빼고 학습시켰습니다.

3.6 가격 로그 변환(log1p)

가격은 저렴한 물건이 대부분이고 아주 비싼 물건은 소수라, 그대로 학습시키면 소수의 비싼 물건에 모델이 너무 민감하게 반응할 수 있습니다. 그래서 가격 대신 $\log_{1p}(\text{price})$ 라는, 가격에 로그를 씌운 값을 학습 목표로 사용했습니다. 이렇게 하면 "1만 원짜리를 9천 원으로 예측"한 것과 "100만 원짜리를 99만 원으로 예측"한 것을 "둘 다 10% 정도 차이"로 비슷하게 다룰 수 있어, 값이 작은 물건과 큰 물건을 더 공평하게 평가할 수 있습니다. 이번에 쓰는 평가 점수(RMSLE)도 로그를 씌운 값끼리의 차이를 재는 방식이라, 학습 목표와 평가 기준이 같은 기준 위에서 일관되게 맞춰집니다.

4. 분석방법(Analysis method)

4.1 모델링 파이프라인 개요

단계	내용
1	카테고리를 "/" 기준으로 대분류·중분류·소분류 3개 항목으로 나눔
2	브랜드·카테고리·상품설명에 빈 칸을 '브랜드없음' 같은 값으로 채움
3	상품 이름 → 단어 등장 횟수, 상품 설명 → 단어 중요도 점수로 숫자화
4	브랜드·상태등급·카테고리 3종 → 있음/없음으로 펼침
5	7개 조각을 이어붙여 최종 표(1,482,535 × 161,568) 완성, shipping은 제외

4.2 사용 모델 설명

Gradient Boosting은 나무(트리) 여러 개를 순서대로 심어가면서, 앞선 나무가 놓친 부분을 다음 나무가 조금씩 보완해 나가는 방식의 인공지능 기법입니다. 나무 한 그루 한 그루는 그리 복잡하지 않지만, 이런 나무를 수십~수백 개 순서대로 쌓으면 꽤 정교한 예측이 가능해집니다.

이번 분석에서는 같은 방식을 두 가지 용도로 썼습니다. 하나는 가격(연속된 숫자)을 맞추는 용도(GradientBoostingRegressor), 다른 하나는 배송비 부담 주체(구매자나 판매자나, 둘 중 하나)를 맞추는 용도(GradientBoostingClassifier)입니다. 두 모델은 "나무를 순서대로 쌓아 오차를 보완한다"는 기본 원리는 같지만, 무엇을 오차로 보고 배우는지가 다릅니다.

설정값	이번에 쓴 값	쉬운 설명
나무 개수(n_estimators)	100	순서대로 심을 나무의 개수
나무 깊이(max_depth)	3	나무 한 그루가 가지를 뻗는 최대 단계 — 얇은 나무를 여러 개 쌓는 방식
학습률(learning_rate)	0.05	나무 한 그루가 최종 결과에 반영되는 비율(조금씩 반영)
시드값(random_state)	156	다시 실행해도 같은 결과가 나오게 고정하는 값
가격 모델의 기준(loss)	squared_error(기본값)	예측과 실제 가격의 차이(제곱)를 줄이는 방향으로 학습
배송비 모델의 기준(loss)	log_loss(기본값)	“구매자일 확률/판매자일 확률”의 오차를 줄이는 방향으로 학습

4.3 모델 복잡도 검증 — AIC·BIC

RMSLE 같은 점수는 “얼마나 잘 맞았는가”만 알려줄 뿐, “그 정확도를 얻으려고 모델이 얼마나 복잡해졌는가”는 알려주지 않습니다. AIC·BIC는 여기에 “너무 복잡한 거 아니야?” 하고 확인하는 별점을 더해 계산하는 점수입니다. 나무를 많이 쓸수록 정확도는 조금씩 좋아지는 경향이 있지만, 나무 개수가 늘어나는 만큼 별점도 함께 늘어나기 때문에, 어느 지점부터는 “더 늘려봐야 실속이 없다”는 신호를 AIC·BIC가 보여줄 수 있습니다.

다만 Gradient Boosting은 원래 선형회귀 같은 통계 모델을 위해 만들어진 AIC·BIC 계산법에 정확히 들어맞는 모델이 아닙니다. “모델이 몇 개의 기준(파라미터)을 배웠는가”를 나무들의 최종 가지 끝(리프) 개수로 근사해서 계산했다는 점을 감안하고 참고용으로만 봐야 합니다.

항목	값
검증용으로 댄 건수	296,507건
오차 제곱의 합(RSS)	약 126,636
기준(파라미터) 근사값 k	801 (전체 리프 개수 800 + 1)
AIC	-250,652.0
BIC	-242,161.5

※ 나무 개수(20/50/100/200)를 바꿔가며 비교하는 실험은 조건 하나를 확인하는 데만 약 3.9시간이 걸려, 이번 보고서 시점까지 모든 조건을 다 끝내지 못했습니다. 준비되는 대로 후속 보고서에 반영할 예정이며,

아래 5.1절의 해석은 나무 100개 조건의 결과만을 근거로 합니다.

BIC가 AIC보다 큰 값으로 나온 것은 데이터 건수가 많을수록 복잡도에 대한 벌점을 더 엄격하게 매기는 BIC 계산식의 구조적 특징 때문입니다. 절대적인 수치 자체보다는, 앞으로 같은 방식으로 계산한 값끼리 비교하는 용도로 활용하는 것이 안전합니다.

4.4 실행환경

항목	값
가격 모델 학습 시간	약 3.9시간(13,993초)
배송비 모델 학습 시간	약 3.9시간(14,066초)
데이터 나누는 방식	80%(학습) : 20%(검증), 배송비 모델은 클래스 비율 유지 (stratify) 적용
검증 전략	단일 홀드아웃(80:20)

나무를 하나씩 순서대로 심어가는 방식이다 보니, 데이터 건수가 많을수록 나무 한 그루를 심는 데 드는 시간도 함께 늘어나 전체 학습 시간이 상당히 오래 걸렸습니다. 그래서 여러 조건을 바꿔가며 반복해서 실험해야 하는 작업은 소규모로 먼저 빠르게 확인해 보고, 최종 확정 단계에서만 전체 데이터로 재확인하는 방식이 현실적으로 더 효율적입니다.

4.5 코드 재사용 및 함수화

오차를 계산하는 함수, 가격을 원래 단위로 되돌리는 함수, 나무의 가지 끝 개수를 세는 함수, 이 세 가지를 처음부터 별도로 만들어 두고 파이프라인 전체에서 그대로 재사용했습니다. "데이터 양이 달라져도 코드를 고치지 않고 그대로 동작하도록" 설계한 덕분에, 분석 과정에서 별도로 손볼 부분이 없었습니다.

5. 결과

5.1 분석 결과

가격 예측 모델과 배송비 부담 주체 예측 모델을 전체 데이터(1,482,535건)로 학습·검증한 결과는 아래와 같습니다.

점수	결과
RMSLE (가격, 낮을수록 좋음)	0.6535

점수	결과
R^2 (가격 설명력, 높을수록 좋음)	0.0906
정확도 (배송비)	0.6707
ROC-AUC (배송비 판별력)	0.7123
Recall 평균 (배송비)	0.6533
F1 평균 (배송비)	0.6517

가격 예측은 RMSLE 0.65 수준으로, 아직 개선의 여지가 있는 결과입니다. R^2 (설명력)도 0.09 정도로 낮게 나와, 브랜드·카테고리·텍스트 정보만으로는 가격 변동의 상당 부분을 설명하기 어렵다는 점을 보여줍니다. 배송비 부담 주체 예측은 정확도 67%, ROC-AUC 0.71 수준으로 무작위 추측보다는 뚜렷하게 나은 판별력을 보였지만, 3.4절에서 확인했듯 판매자부담 케이스를 상대적으로 더 자주 놓치는 한계가 있습니다.

AIC·BIC는 나무 100개 조건에서 계산했습니다(AIC -250,652.0, BIC -242,161.5). 나무 개수를 20/50/100/200으로 바꿔가며 비교하는 실험은 4.3절에서 밝힌 대로 계산 시간 문제로 아직 다 마치지 못해, 이번 결과만 가지고 “지금 설정이 통계적으로 과하게 복잡한지”까지는 결론 내리지 않았습니다.

5.2 후속 검토 항목

- 나무 개수(20/50/100/200)를 비교하는 실험과 관련 검증 — 계산 시간 문제로 아직 끝내지 못해, 후속 분석에서 이어서 진행할 예정
- 전체 데이터 학습 시간(약 3.9 시간)이 실무에서 쓰기엔 부담스러운 수준이라, 대용량 처리에 더 유리한 방식(HistGradientBoosting 계열)으로 바꾸면 시간이 얼마나 줄어드는지 확인
- 판매자부담 배송비를 잘 못 맞추는 문제를 개선할 수 있는지 확인 — 클래스 불균형 보완, 샘플 가중치 조정 등을 검토
- 설정값을 자동으로 더 정교하게 찾아주는 방법(Hyperopt/Optuna 등) 적용 검토